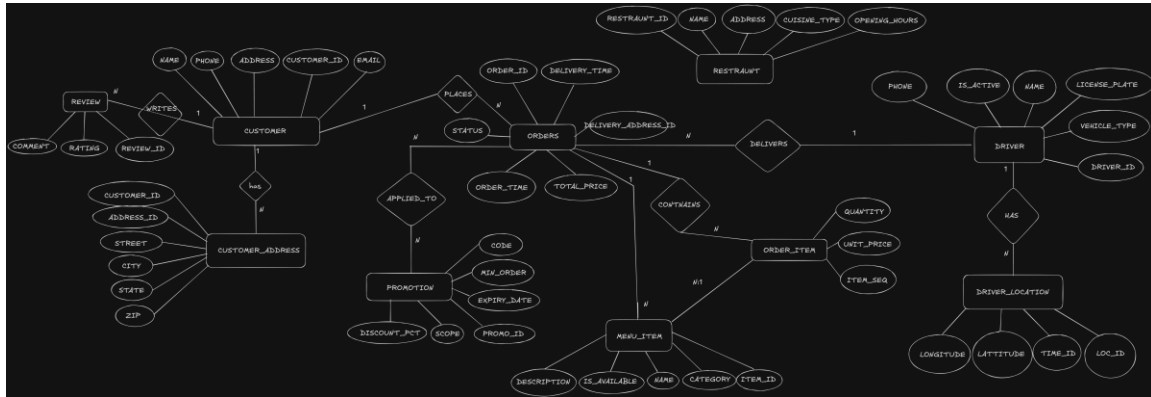


Part 1: Assumptions

- Each order belongs to exactly one customer.
- Each order is placed with exactly one restaurant.
- A customer can have multiple saved delivery addresses.
- Each menu item is stored without a separate category entity.
- Menu prices can change over time, so order items store the price at purchase time.
- An order can contain multiple menu items.
- A driver can deliver many orders, but each order has at most one driver.
- A driver may not be assigned when an order is first created.
- Driver location is stored as time-stamped history records.
- Customers can leave multiple reviews.
- Promotions apply only to orders (not restaurants or categories).
- An order can use zero or one promotion code.
- Only completed orders can be reviewed.
- Restaurants remain in the system even if inactive.

Part 2: ER Diagram



Entities: CUSTOMER, CUSTOMER_ADDRESS, ORDERS, RESTAURANT, DRIVER, DRIVER_LOCATION, MENU_ITEM, ORDER_ITEM, PROMOTION, REVIEW.

Relationships:

- CUSTOMER has CUSTOMER_ADDRESS (1:N)
- CUSTOMER places ORDERS (1:N)
- ORDERS contains ORDER_ITEM (1:N)
- ORDER_ITEM references MENU_ITEM (N:1)
- DRIVER delivers ORDERS (1:N)
- DRIVER has DRIVER_LOCATION (1:N)
- ORDERS applied_to PROMOTION (N:1, optional)
- CUSTOMER writes REVIEW (1:N)

Part 3: Decision Log

1. One address vs multiple addresses

Option 1: Allow only one address per customer

Option 2: Allow multiple addresses per customer

Chosen: Option 2

I chose to allow multiple addresses because in a real-world food delivery system, customers often order from different locations such as home, work, or a friend's house. Limiting users to one address would reduce usability and make the system less practical. By creating a separate CUSTOMER_ADDRESS entity, the database remains normalized and avoids repeating address fields in the CUSTOMER table. This design also allows flexibility for future features like address labeling (for example, "Home" or "Work").

2. Store only current price vs store historical price

Option 1: Store only the current price in MENU_ITEM

Option 2: Store the price at the time of purchase in ORDER_ITEM

Chosen: Option 2

I chose to store the price at the time of purchase in ORDER_ITEM because menu prices can change over time. If only the current price were stored, past orders would become inaccurate when prices change. By storing the unit_price in ORDER_ITEM, the system preserves the exact price the customer paid at checkout. This ensures accurate receipts, reporting, and auditing, which are important for both customers and the business.

3. Reviews tied to customer vs tied to order

Option 1: Link reviews directly to orders

Option 2: Link reviews directly to customers

Chosen: Option 2

I chose to link reviews directly to customers to match the ER diagram and keep the structure simpler. While tying reviews to orders could provide more detailed tracking, it would add complexity to the schema. Since the main goal is to capture customer

feedback, associating reviews with customers still provides useful information while keeping the design clean and easy to manage.

4. Single location field vs location history tracking

Option 1: Store only the current location in DRIVER

Option 2: Store multiple time-stamped records in DRIVER_LOCATION

Chosen: Option 2

I chose to store driver locations in a separate DRIVER_LOCATION table with timestamps because the system needs to track movement during deliveries. A single location field would only show the current position and lose all historical data. By storing multiple records, the system can track routes, estimate delivery times, and analyze driver performance. This also supports future features like real-time tracking and route optimization.

5. Promotions applied broadly vs only to orders

Option 1: Allow promotions to apply to restaurants and categories

Option 2: Allow promotions to apply only to orders

Chosen: Option 2

I chose to simplify promotions so they apply only to orders because it keeps the database design more straightforward and matches the ER diagram. While more complex promotion logic could support different scopes, it would require additional tables and relationships. For this design, applying promotions at the order level still meets the requirements while reducing complexity and making implementation easier.

Part 4: Relational Tables

- CUSTOMER(customer_id PK, name, email, phone)
- CUSTOMER_ADDRESS(address_id PK, customer_id FK, street, city, state, zip)
- RESTAURANT(restaurant_id PK, name, address, cuisine_type, opening_hours)
- ORDERS(order_id PK, customer_id FK, restaurant_id FK, driver_id FK NULLABLE, order_time, delivery_time, status, total_price)
- DRIVER(driver_id PK, name, phone, vehicle_type, license_plate, is_active)
- DRIVER_LOCATION(loc_id PK, driver_id FK, latitude, longitude, time_id)
- MENU_ITEM(item_id PK, name, description, is_available)
- ORDER_ITEM(order_id PK/FK, item_seq PK, item_id FK, quantity, unit_price)
- PROMOTION(promo_id PK, code, discount_pct, min_order, expiry_date)
- REVIEW(review_id PK, customer_id FK, rating, comment)

Part 5: Normalization Check

1NF: All attributes are atomic.

2NF: All non-key attributes depend on the full primary key.

3NF: No transitive dependencies exist in the design.

Part 6: Subscription Model

To support a subscription model, I would extend the database by introducing new tables instead of modifying the CUSTOMER table directly. This approach keeps the design flexible and scalable.

New Tables:

SUBSCRIPTION_PLAN

- plan_id (PK)
- plan_name
- monthly_fee
- free_delivery_flag
- priority_support_flag
- active_status

CUSTOMER_SUBSCRIPTION

- customer_subscription_id (PK)
- customer_id (FK → CUSTOMER)
- plan_id (FK → SUBSCRIPTION_PLAN)
- start_date
- end_date
- subscription_status
- auto_renew

Explanation:

I chose to create separate subscription tables instead of adding a simple “premium” column to CUSTOMER because subscriptions can change over time. Customers may upgrade, downgrade, cancel, or reactivate their plans. By storing subscriptions in a separate table, the system can track this history instead of overwriting data.

This design also allows the platform to support multiple subscription plans in the future, such as basic, premium, or business tiers. Each plan can include different benefits like free delivery or priority support without modifying the core CUSTOMER table.

Additionally, keeping subscription data separate improves normalization and avoids unnecessary null values in the CUSTOMER table. It also makes it easier to manage billing, renewals, and eligibility checks. For example, when an order is placed, the system can check if the customer has an active subscription and automatically set the delivery fee to zero if applicable.

Overall, this approach provides flexibility, scalability, and better long-term maintainability for the system.

Conclusion

This design is meant to balance realism, clear business rules, and normalization. The schema supports customers, restaurants, menus, promotions, drivers, delivery tracking, and customer feedback while still being flexible enough for future growth.